

Contenido

● 1	Presentación del proyecto	1
● 2	Stack tecnológico y arquitectura	3
● 3	Requisitos técnicos comunes	6
● 4	Fases y temporalización	9
● 5	Caso 1 · CoffeeStation	11
● 6	Caso 2 · FitZone	15
● 7	Caso 3 · CineTown	19
● 8	Caso 4 · ParkControl	23
● 9	Contratos de integración	27

Cómo usar este documento

Lee completas las secciones **1 a 4** y la **9**: son comunes para todo el grupo. De las secciones 5 a 8 solo necesitas la de tu caso y la del caso con el que te integras.

Cada caso sigue la misma estructura: reunión con el cliente, roles, requisitos funcionales (RF), reglas de negocio (RN), entidades mínimas, los 16 endpoints, sugerencias de consultas e integración.

1 Presentación del proyecto

Vas a desarrollar, de forma individual, el backend completo de un negocio real: desde el diseño de su base de datos en PostgreSQL hasta una API REST segura con Spring Boot que, en la fase final, se comunica con la API de un compañero.

1.1 El ecosistema SafaReyes

SafaReyes es un complejo de ocio y servicios de nueva apertura. En su interior conviven cuatro negocios independientes que han decidido digitalizarse al mismo tiempo y colaborar entre sí para fidelizar a sus clientes.



Cada negocio tiene su propio sistema, desarrollado por ti de forma independiente. Pero los cuatro forman una **cadena de colaboración**: cada API **consume** un servicio de otra y **expone** un servicio para otra. La flecha va del sistema que llama (cliente HTTP) al que responde (servidor). Así, todos los casos hacen el mismo trabajo de integración: un endpoint expuesto y un cliente HTTP.

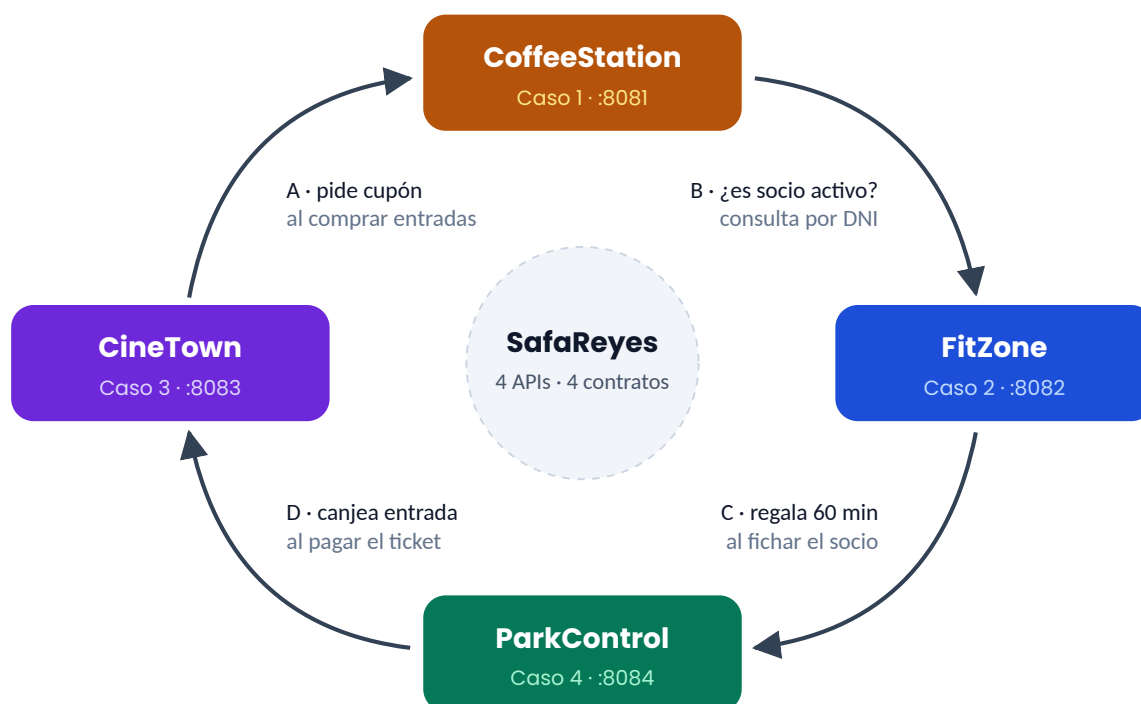


Figura 1. Cadena de integración del ecosistema SafaReyes

1.2 Dinámica de trabajo

1. El profesor te asigna uno de los cuatro casos. Si hay más alumnos que casos, varios alumnos desarrollarán el mismo caso de forma independiente.

2. Actúas como desarrollador principal: eres responsable del diseño de datos, la lógica de negocio, la seguridad y la API.
3. El enunciado de tu caso está escrito como lo contaría el cliente. Tu primera tarea es **analizarlo**: extraer entidades, atributos, relaciones y reglas. La base de datos no se te da hecha.
4. En la Fase 5 el profesor te asigna un **compañero de integración** del caso al que tu API llama. Los contratos (URL, JSON, cabeceras) están fijados en la sección 9 y son idénticos para todos, así que puedes desarrollar tu cliente antes de que tu compañero termine.
5. El trabajo es individual. Puedes consultar documentación y resolver dudas con compañeros, pero el código y el diseño son tuyos.

1.3 Qué debes leer

Las secciones 2, 3, 4 y 9 son comunes y obligatorias para todos. De las secciones 5 a 8 solo necesitas la de tu caso, aunque te conviene leer también la del caso con el que te integras.

2

Stack tecnológico y arquitectura

Todos los proyectos usan exactamente el mismo stack. No se admiten sustituciones sin autorización previa del profesor.

2.1 Tecnologías obligatorias

Área	Tecnología	Uso en el proyecto
Lenguaje	Java 21	Records para DTOs, <code>switch</code> moderno, <code>Optional</code>
Framework	Spring Boot 3.x (última estable)	Proyecto generado con Spring Initializr y Maven
Base de datos	PostgreSQL 16	Arrancada con Docker Compose (<code>compose.yaml</code> en el repositorio)
Persistencia	Spring Data JPA + Hibernate	Entidades, relaciones y repositorios
Validación	Jakarta Bean Validation	<code>@Valid</code> , <code>@NotBlank</code> , <code>@Positive</code> , <code>@Email</code> , validadores propios
Boilerplate	Lombok	Entidades, servicios, logging
Mapeo	MapStruct	Conversión entidad ↔ DTO, sin mapeo manual
Seguridad	Spring Security 6 + JWT	Autenticación sin estado y autorización por roles
Cliente HTTP	<code>RestClient</code> de Spring	Consumo de la API del compañero
Documentación	springdoc-openapi (Swagger UI)	Documentación viva de todos los endpoints
Pruebas manuales	Postman o Bruno	Colección exportada en el repositorio
Control de versiones	Git + GitHub	Repositorio del proyecto

Dependencias de Initializr: *Spring Web*, *Spring Data JPA*, *PostgreSQL Driver*, *Validation*, *Spring Security*, *Lombok*, *Spring Boot DevTools*. MapStruct, JWT y springdoc se añaden a mano en el `pom.xml`.

Importante Lombok debe ejecutarse antes que MapStruct. Configura `maven-compiler-plugin` con `lombok`, `lombok-mapstruct-binding` y `mapstruct-processor` en `annotationProcessorPaths`.

2.2 Arquitectura en capas

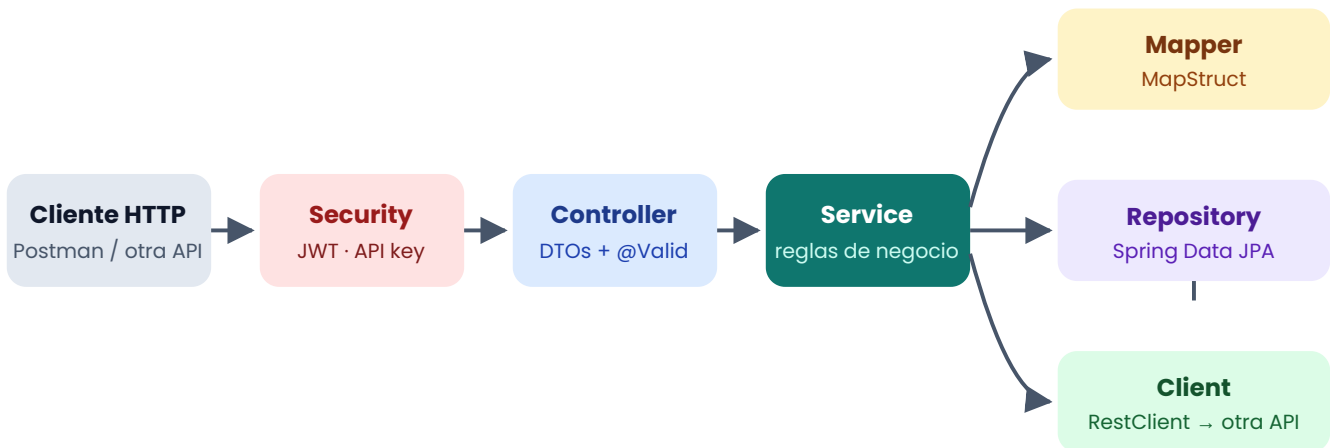


Figura 2. Capas de la aplicación y sentido de las dependencias

- Un controlador nunca accede a un repositorio ni devuelve una entidad. Recibe y devuelve DTOs.
- Toda la lógica de negocio vive en la capa de servicio. Las operaciones que escriben usan `@Transactional`.
- La inyección de dependencias es siempre por constructor (`@RequiredArgsConstructor` + campos `final`). Prohibido `@Autowired` en atributos.
- Las llamadas a la API externa se encapsulan en una clase de la capa `client`. El servicio no conoce URLs ni cabeceras.

2.3 Estructura de paquetes

PAQUETES

```
es.safareyes.<caso>
├── config          → RestClient, OpenAPI, propiedades
├── controller      → @RestController
├── dto
│   ├── request    → records de entrada
│   └── response    → records de salida
├── entity          → @Entity y enums
├── exception       → excepciones propias + @RestControllerAdvice
├── mapper          → interfaces @Mapper (MapStruct)
├── repository      → interfaces JpaRepository
├── security        → SecurityConfig, filtro JWT, UserDetailsService
├── service         → interfaces + implementaciones
└── client         → cliente de integración con la otra API
```

Sustituye `<caso>` por `coffeestation`, `fitzone`, `cinetown` o `parkcontrol`. Nombre recomendado del repositorio: `safareyes-<caso>-<tu-apellido>`.

2.4 Convenciones

- Tablas y columnas en `snake_case` y plural para tablas (`lineas_pedido`). Clases en `PascalCase`, atributos en `camelCase`.
- Todas las rutas empiezan por `/api/v1`, usan sustantivos en plural y minúsculas con guiones (`/informes/ventas-dia`).
- Importes con `BigDecimal` en Java y `NUMERIC(10,2)` en PostgreSQL. Nunca `double` para dinero.
- Fechas con `LocalDate`, `LocalDateTime` y zona `Europe/Madrid`.

- Configuración sensible (contraseña de BD, secreto JWT, API keys) en variables de entorno, nunca en el código subido a GitHub.
- Commits pequeños y con mensajes descriptivos.

3

Requisitos técnicos comunes

Estos requisitos son iguales en los cuatro casos. En la ficha de cada caso encontrarás ejemplos concretos de dónde aplicarlos.

3.1 Base de datos

- Mínimo **6 tablas de dominio** más las tablas de seguridad (`usuarios` y, si lo necesitas, `roles`).
- Modelo normalizado hasta 3FN. Justifica en el README cualquier desnormalización (por ejemplo, guardar el precio aplicado en una línea).
- Al menos una relación **1:N**, una **N:M** con tabla intermedia y una relación opcional (FK que admite `NULL`).
- Restricciones declaradas en el script: `PRIMARY KEY` , `FOREIGN KEY` con su `ON DELETE` , `UNIQUE` , `NOT NULL` , `CHECK` (importes positivos, rangos, estados válidos).
- Al menos un índice creado a mano sobre una columna por la que se busca con frecuencia.
- `data.sql` con datos suficientes para probar todos los endpoints: mínimo 10 filas en las tablas principales y casos límite (agotados, caducados, inactivos, completos).

3.2 Entidades y Lombok

- Anotaciones Lombok permitidas en entidades: `@Getter` , `@Setter` , `@NoArgsConstructor` , `@AllArgsConstructor` , `@Builder` , `@ToString(exclude = ...)` .
- **Prohibido `@Data` en entidades:** su `equals` , `hashCode` y `toString` recorren las relaciones y provocan bucles y cargas innecesarias. Debes saber explicar por qué.
- En servicios y controladores: `@RequiredArgsConstructor` y `@Slf4j` . Registra en log las operaciones de negocio relevantes y los fallos de integración.
- Los enums se persisten con `@Enumerated(EnumType.STRING)` .
- Relaciones con `FetchType.LAZY` por defecto. Usa `EAGER` solo si lo justificas.
- Al menos una entidad con campos de auditoría (`creadoEn` , `actualizadoEn`) rellenos automáticamente con `@PrePersist` / `@PreUpdate` o con JPA Auditing.

3.3 Formatos de consulta en repositorios

Este es el núcleo de la Fase 2. Tus repositorios deben contener, como mínimo, las consultas de esta tabla, y todas deben usarse desde algún servicio.

#	Formato	Mín.	Qué debe demostrar
Q1	Métodos derivados del nombre (<i>query methods</i>)	5	Combinar <code>And</code> / <code>Or</code> , <code>Between</code> , <code>LessThan</code> / <code>GreaterThan</code> , <code>Containing</code> / <code>IgnoreCase</code> , <code>OrderBy</code> , <code>existsBy</code> , <code>countBy</code> , <code>findFirst</code> / <code>Top</code>
Q2	JPQL con <code>@Query</code> y parámetros con nombre (<code>@Param</code>)	3	Al menos una con <code>JOIN</code> entre entidades y una con función de agregado (<code>SUM</code> , <code>COUNT</code> , <code>AVG</code>) y <code>GROUP BY</code>
Q3	SQL nativo con <code>@Query(nativeQuery = true)</code>	2	Consultas de informe con funciones propias de PostgreSQL (<code>DATE_TRUNC</code> , <code>EXTRACT</code> , <code>COALESCE</code> ...)
Q4	Consultas de modificación con <code>@Modifying</code>	1	<code>UPDATE</code> o <code>DELETE</code> masivo, dentro de una transacción, con <code>clearAutomatically = true</code>

#	Formato	Mín.	Qué debe demostrar
Q5	Proyecciones	2	Una proyección por interfaz y una por clase/record con <code>SELECT new ...</code> en JPQL
Q6	Paginación y ordenación	1	Parámetro <code>Pageable</code> y retorno <code>Page<...></code> , expuesto en un endpoint con <code>?page=&size=&sort=</code>
Q7	Consultas dinámicas	1	<code>JpaSpecificationExecutor</code> + <code>Specification</code> para filtros opcionales combinables
Q8	Evitar el problema N+1	1	<code>@EntityGraph</code> o <code>JOIN FETCH</code> en una consulta que cargue una entidad con su colección

16 consultas propias como mínimo. En el README incluye una tabla con cada consulta, su formato (Q1-Q8), el repositorio en que está y el endpoint que la usa.

3.4 DTOs y MapStruct

- Todos los DTOs son `record` de Java, separados en `dto.request` y `dto.response`.
- Cada recurso tiene como mínimo un DTO de creación, uno de respuesta y, si se puede modificar, uno de actualización. Los DTOs de respuesta nunca incluyen contraseñas ni datos internos.
- Validaciones de entrada en los DTOs de petición, nunca en las entidades. Al menos un **validador personalizado** (anotación propia + `ConstraintValidator`), por ejemplo para DNI, matrícula o código de cupón.
- Mapeo exclusivamente con MapStruct (`@Mapper(componentModel = "spring")`).

Los mappers deben demostrar:

- `@Mapping(target = ..., source = ...)` con campos de nombre distinto o anidados (`source = "sala.numero"`).
- `@Mapping(target = ..., ignore = true)` para campos que calcula el servicio.
- Actualización parcial con `@MappingTarget` y `NullValuePropertyMappingStrategy.IGNORE`.
- Mapeo de colecciones (lista de entidades a lista de DTOs) y uso de otro mapper con `uses = ...`.

3.5 Seguridad

- Autenticación **sin estado** con JWT. Endpoints comunes a todos los casos: `POST /api/v1/auth/registro` (alta de un usuario con rol de cliente, socio o abonado) y `POST /api/v1/auth/login` (devuelve el token JWT, que caduca en 60 minutos).
- Contraseñas cifradas con `BCryptPasswordEncoder`.
- Tres roles por caso (definidos en tu ficha). Los usuarios con roles de personal se crean desde `data.sql` o por un administrador.
- Autorización en dos niveles: reglas por ruta en el `SecurityFilterChain` y `@PreAuthorize` en al menos tres métodos.
- Al menos una regla de **propiedad del recurso**: un cliente solo puede consultar sus propios pedidos, compras, cuotas o tickets.
- Los endpoints de integración (sección 9) no usan JWT: se protegen con la cabecera `X-API-KEY`, comprobada por un filtro propio. La clave se lee de una variable de entorno.
- Swagger UI y `/api/v1/auth/**` son públicos. CSRF desactivado por ser una API sin estado.

3.6 Gestión de errores y códigos HTTP

Todos los errores se devuelven con el mismo formato, basado en `ProblemDetail` (RFC 9457), generado desde un único `@RestControllerAdvice`. Crea al menos cuatro excepciones propias (por ejemplo `RecursoNoEncontradoException`, `ReglaNegocioException`, `ConflictoException`, `IntegracionException`).

```
JSON
{
  "type": "about:blank",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "No existe el producto con id 57",
  "instance": "/api/v1/productos/57",
  "timestamp": "2026-10-14T09:31:02"
}
```

Código	Cuándo usarlo
200 OK	Consulta o modificación correcta
201 Created	Recurso creado, con cabecera <code>Location</code>
204 No Content	Borrado (lógico o físico) correcto
400 Bad Request	Validación de DTO fallida (incluye la lista de campos erróneos)
401 Unauthorized	Sin token, token inválido o caducado, API key incorrecta
403 Forbidden	Token válido pero rol insuficiente o recurso ajeno
404 Not Found	El recurso no existe
409 Conflict	Duplicados o estado incompatible (DNI repetido, cupón ya canjeado)
422 Unprocessable Content	La petición es correcta pero viola una regla de negocio (sesión completa, sin plazas)
503 Service Unavailable	Solo en el cliente de integración, si la otra API no responde y la operación no puede continuar

3.7 Documentación y calidad

- Swagger UI accesible en `/swagger-ui.html` con cada endpoint descrito (`@Operation`, `@Tag`) y el esquema de seguridad Bearer configurado.
- README con: descripción, instrucciones para arrancar con Docker Compose, variables de entorno, diagrama E/R, usuarios de prueba, tabla de consultas (3.3) y decisiones de diseño.
- Colección de Postman o Bruno con todas las peticiones, incluidos casos de error.
- Pruebas automáticas (recomendado): tests de repositorio con `@DataJpaTest` y de servicio con JUnit 5 + Mockito.
- El proyecto debe arrancar con `docker compose up -d` y `./mvnw spring-boot:run` siguiendo solo el README.

4

Fases y temporalización

El proyecto se organiza en **10 semanas** repartidas en 7 fases (de la 0 a la 6). Las fechas concretas y el procedimiento de entrega de cada fase los indicará el profesor.

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
F0	Análisis									
F1	Base de datos									
F2			Persistencia JPA							
F3					API REST					
F4							Seguridad			
F5								Integración		
F6										Cierre

Fase	Semanas	Contenido	Resultado esperado
0 · Arranque y análisis	1	Lectura del caso, repositorio, Docker Compose	Proyecto creado y conectado a PostgreSQL
1 · Base de datos	1-2	Modelo E/R, modelo relacional, scripts	<code>schema.sql</code> , <code>data.sql</code> , diagrama E/R y documento de análisis
2 · Persistencia JPA	3-4	Entidades, relaciones, repositorios y consultas	Entidades mapeadas y las 16 consultas funcionando
3 · API REST	5-6	Servicios, DTOs, MapStruct, controladores, errores	Los 16 endpoints de negocio de tu caso
4 · Seguridad	7	Usuarios, JWT, roles, API key	API protegida según la matriz de roles de tu caso
5 · Integración	8-9	Endpoint expuesto y cliente <code>RestClient</code>	Integración probada con tu compañero
6 · Cierre	10	Revisión final y documentación	Proyecto completo y documentado

Lista de tareas por fase

Fase 0 · Arranque y análisis

- ☐ Crear el repositorio Git `safareyes-<caso>-<tu-apellido>`.
- ☐ Generar el proyecto en Spring Initializr con las dependencias de la sección 2.1.
- ☐ Añadir `compose.yaml` con PostgreSQL 16 y comprobar que la aplicación conecta.
- ☐ Leer tu caso y subrayar sustantivos (posibles entidades), verbos (operaciones) y condiciones (reglas de negocio).

Fase 1 · Base de datos

- ☐ Documento `docs/analisis.md`: entidades, atributos, relaciones con su cardinalidad y reglas de negocio que afectan a los datos.
- ☐ Diagrama E/R (draw.io, dbdiagram.io o similar) exportado en `docs/`.
- ☐ `schema.sql` con todas las restricciones de la sección 3.1.
- ☐ `data.sql` con datos realistas y casos límite.
- ☐ Tres consultas SQL de ejemplo en `docs/consultas.sql` que respondan a preguntas del cliente (base para las consultas nativas).

En esta fase la base de datos se crea con los scripts. A partir de la Fase 2 configura `spring.jpa.hibernate.ddl-auto=validate` para que Hibernate compruebe que tus entidades coinciden con tu esquema.

Fase 2 · Persistencia JPA

- ☐ Una entidad por tabla de dominio, con relaciones bidireccionales donde tenga sentido (`mappedBy`).
- ☐ Enums del dominio y campos de auditoría (sección 3.2).
- ☐ Repositorios con las 16 consultas mínimas de la sección 3.3.
- ☐ `CommandLineRunner` temporal o tests `@DataJpaTest` que demuestren que cada consulta funciona.

Fase 3 · API REST

- ☐ DTOs y mappers MapStruct según la sección 3.4.
- ☐ Servicios con todas las reglas de negocio (RN) de tu caso.
- ☐ Controladores con los 16 endpoints de negocio de tu caso, códigos HTTP correctos y paginación.
- ☐ `@RestControllerAdvice` y excepciones propias (sección 3.6).
- ☐ Swagger UI funcionando.

Fase 4 · Seguridad

- ☐ Tabla `usuarios`, entidad y `UserDetailsService`.
- ☐ Registro, login y filtro JWT.
- ☐ Matriz de roles de tu caso aplicada y regla de propiedad del recurso.
- ☐ Filtro de API key para `/api/v1/integracion/**`.

Fase 5 · Integración

- ☐ Implementar el endpoint que **expones** según el contrato de la sección 9.
- ☐ Implementar el cliente `RestClient` que **consume** la API de tu compañero, con URL y API key en `application.yml` leídas de variables de entorno.
- ☐ Tiempo de espera máximo de 3 segundos y tratamiento de errores (4xx, 5xx, sin respuesta).
- ☐ Desarrollar primero contra un simulador y después contra la API real del compañero.
- ☐ Prueba conjunta con tu compañero documentada en `docs/integracion.md`.

Fase 6 · Cierre

- ☐ README completo, colección Postman/Bruno y Swagger actualizados.
- ☐ Repasar los requisitos de las secciones 3 y de tu caso y comprobar que se cumplen todos.
- ☐ Comprobar que el proyecto arranca desde cero siguiendo solo el README.